



Level Averages in a Binary Tree (easy)

We'll cover the following ^

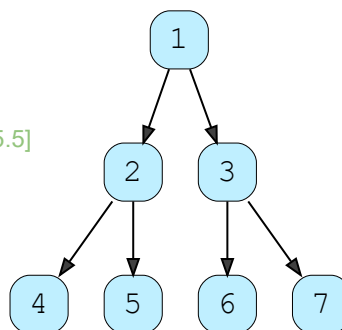
- Problem Statement
- Try it yourself
- Solution
- Code
- Time complexity
- Space complexity
- Similar Problems

Problem Statement

Given a binary tree, populate an array to represent the **averages of all of its levels**.

Example 1:

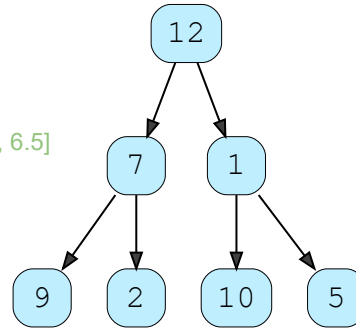
Level Averages: [1, 2.5, 5.5]



Example 2:



Level Averages: [12.0, 4.0, 6.5]



Try it yourself

Try solving this question here:

Java

Python3

JS

C++

```
1 import java.util.*;
2
3 class TreeNode {
4     int val;
5     TreeNode left;
6     TreeNode right;
7
8     TreeNode(int x) {
9         val = x;
10    }
11 };
12
13 class LevelAverage {
14     public static List<Double> findLevelAverages(TreeNode root) {
15         List<Double> result = new ArrayList<>();
16         // TODO: Write your code here
17         return result;
18     }
19
20     public static void main(String[] args) {
21         TreeNode root = new TreeNode(12);
22         root.left = new TreeNode(7);
23         root.right = new TreeNode(1);
24         root.left.left = new TreeNode(9);
25         root.left.right = new TreeNode(2);
26         root.right.left = new TreeNode(10);
27         root.right.right = new TreeNode(5);
28         List<Double> result = LevelAverage.findLevelAverages(root);
29         System.out.print("Level averages are: " + result);
30     }
31 }
32
```

Run

Save

Reset

Tr

Solution

This problem follows the [Binary Tree Level Order Traversal](#) pattern. We can follow the same **BFS** approach. The only difference will be that instead of keeping track of all nodes of a level, we will only track the running sum of the values of all nodes in each level. In the end, we will append the average of the current level to the result array.

> Code

Here is what our algorithm will look like; only the highlighted lines have changed:



```

1  import java.util.*;
2
3  class TreeNode {
4      int val;
5      TreeNode left;
6      TreeNode right;
7
8      TreeNode(int x) {
9          val = x;
10     }
11 };
12
13 class LevelAverage {
14     public static List<Double> findLevelAverages(TreeNode root) {
15         List<Double> result = new ArrayList<>();
16         if (root == null)
17             return result;
18
19         Queue<TreeNode> queue = new LinkedList<>();
20         queue.offer(root);
21         while (!queue.isEmpty()) {
22             int levelSize = queue.size();
23             double levelSum = 0;
24             for (int i = 0; i < levelSize; i++) {
25                 TreeNode currentNode = queue.poll();
26                 // add the node's value to the running sum
27                 levelSum += currentNode.val;
28                 // insert the children of current node to the queue
29                 if (currentNode.left != null)
30                     queue.offer(currentNode.left);
31                 if (currentNode.right != null)
32                     queue.offer(currentNode.right);
33             }
34             // append the current level's average to the result array
35             result.add(levelSum / levelSize);
36         }
37
38         return result;
39     }
40
41     public static void main(String[] args) {
42         TreeNode root = new TreeNode(12);
43         root.left = new TreeNode(7);
44         root.right = new TreeNode(1);
45         root.left.left = new TreeNode(9);
46         root.left.right = new TreeNode(2);
47         root.right.left = new TreeNode(10);
48         root.right.right = new TreeNode(5);
49         List<Double> result = LevelAverage.findLevelAverages(root);
50         System.out.print("Level averages are: " + result);
51     }
52 }
53

```

Run

Save

Reset



Time complexity

The time complexity of the above algorithm is $O(N)$, where 'N' is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space complexity

The space complexity of the above algorithm will be $O(N)$ which is required for the queue. Since we can have a maximum of $N/2$ nodes at any level (this could happen only at the lowest level), therefore we will need $O(N)$ space to store them in the queue.

Similar Problems

Problem 1: Find the largest value on each level of a binary tree.

Solution: We will follow a similar approach, but instead of having a running sum we will track the maximum value of each level.

```
maxValue = max(maxValue, currentNode.val)
```

